

Registers in Computer Architecture

1. Introduction

What is a Register?

A **Register** is a small, fast storage location inside the CPU that holds data, addresses, or instructions temporarily during processing. Registers are crucial for instruction execution and data manipulation in computer systems.

Registers are made from **flip-flops** and can typically store one word (e.g., 32 or 64 bits). Unlike RAM, they are directly accessible by the processor with virtually no delay.

2. Role of Registers in CPU

- Store operands and intermediate results during ALU operations.
- Hold memory addresses during load/store operations.
- Serve as counters or pointers (e.g., Program Counter, Stack Pointer).
- Fast access for control units to maintain status flags and instructions.

3. Construction of Registers

Registers are constructed using:

- **Flip-flops**: Each bit in a register is stored in a D flip-flop.
- **Multiplexers (MUX)**: For selecting which register to read/write.
- **Decoders**: Used for selecting specific registers during writing.
- **Control signals**: Enable read/write and clock synchronization.

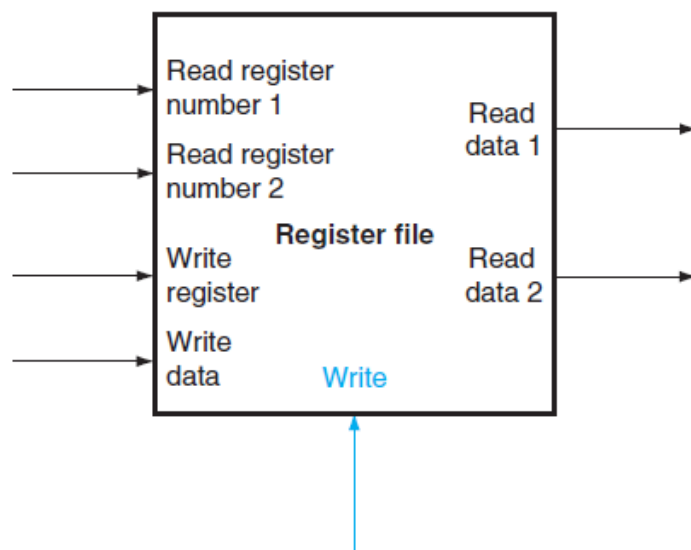
4. Types of Registers

Register Type	Description
Data Registers	Store general-purpose data during instruction execution.
Address Registers	Store memory addresses to access RAM.
Instruction Register (IR)	Holds the current instruction being decoded.
Program Counter (PC)	Holds address of the next instruction.
Stack Pointer (SP)	Points to the top of the stack.
Status Register / Flags	Holds condition codes (Zero, Carry, Overflow, etc.).

5. Register File Architecture

A **register file** is a set of registers connected via multiplexers and decoders for simultaneous read and write access.

- Typically supports **2 read ports** and **1 write port**.
- Each register is addressed by a binary number (index).
- Read and write operations are performed in a single clock cycle.



6. Read and Write Operations

Read Operation

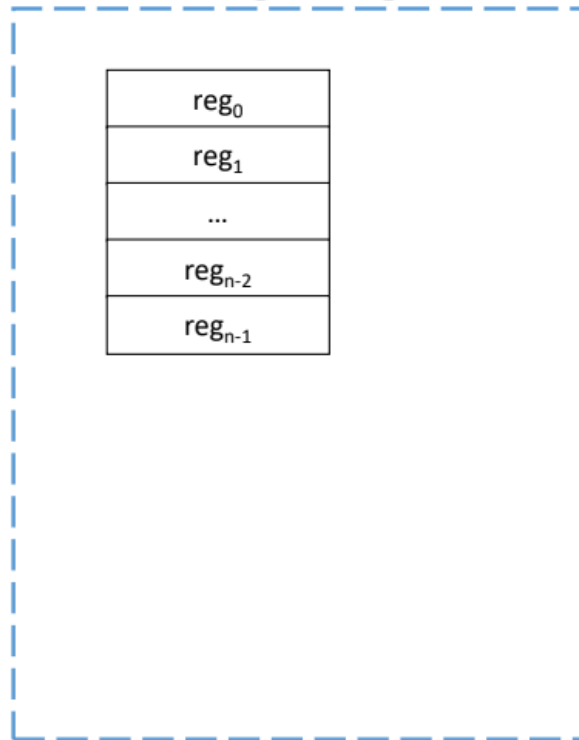
- Control unit sets the **read register** address.

- The value in the addressed register is output via the read port.
- No change occurs to the stored data during a read.

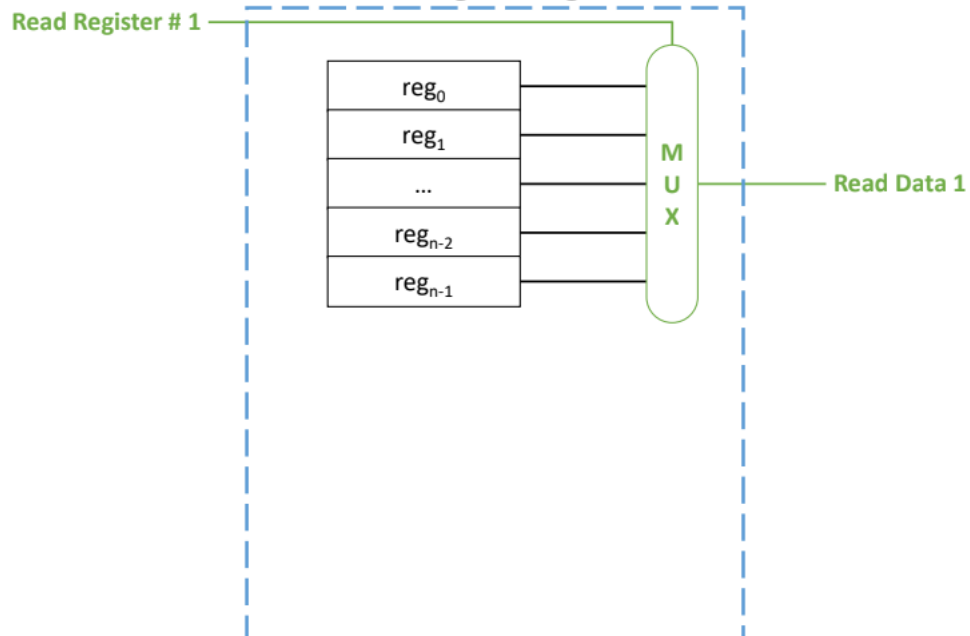
Reading Registers



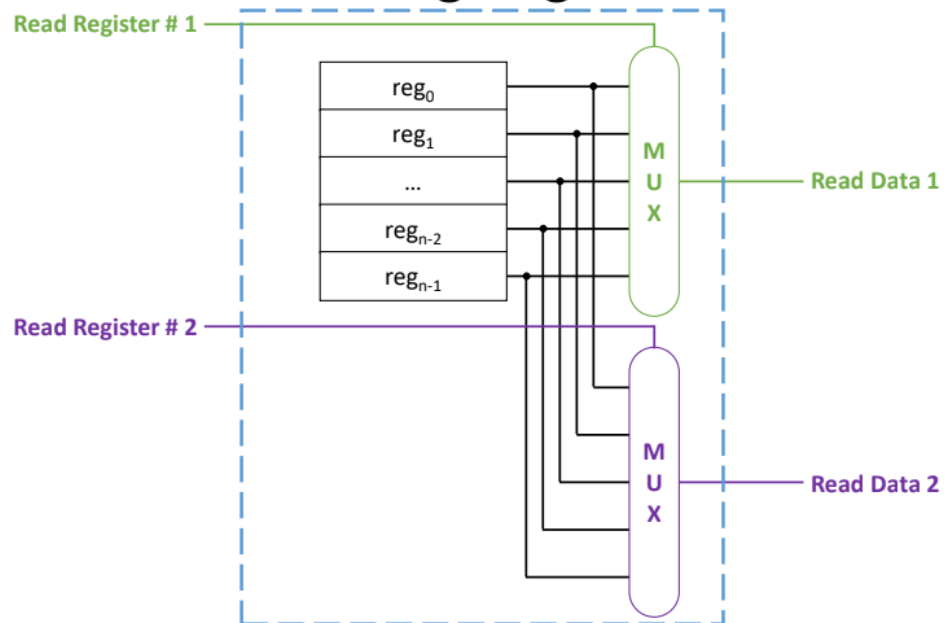
Reading Registers



Reading Registers



Reading Registers



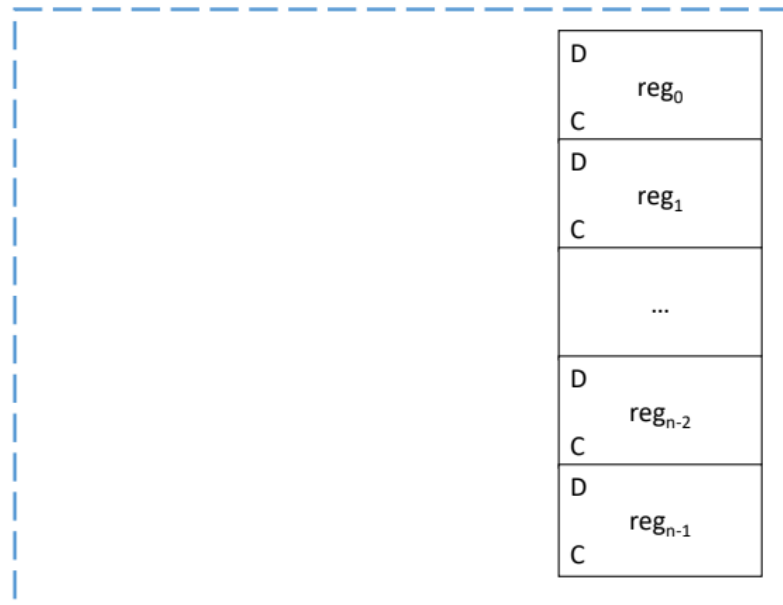
Write Operation

- Control unit sets the **write register** address and **write data**.
- The **write enable** signal is set to 1.
- On clock edge, data is stored into the selected register.

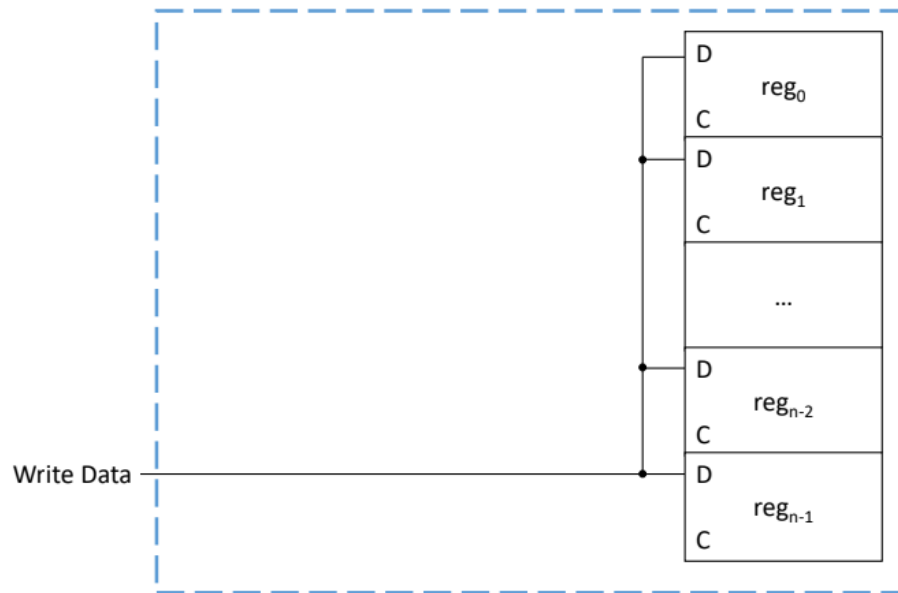
Writing Registers



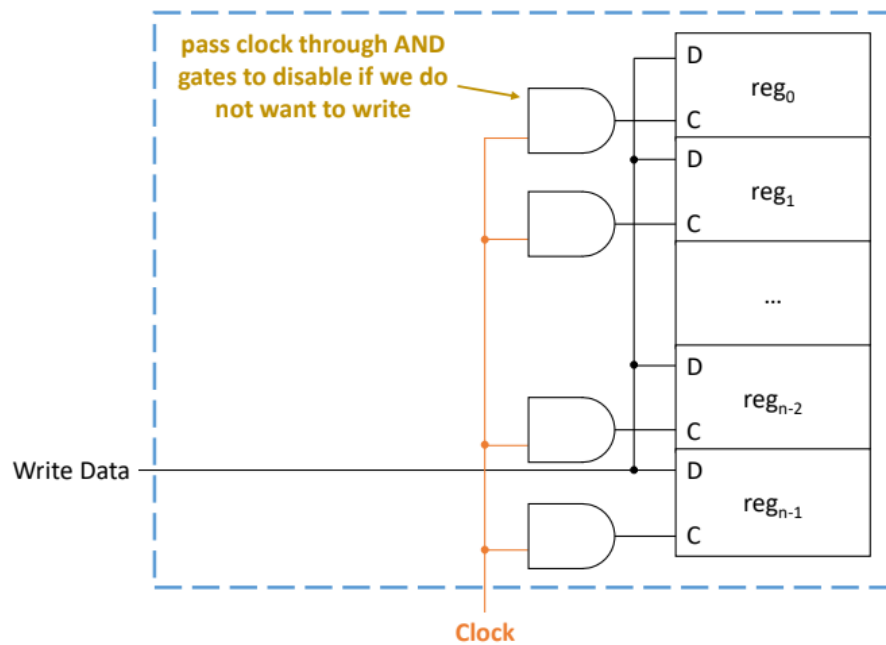
Writing Registers



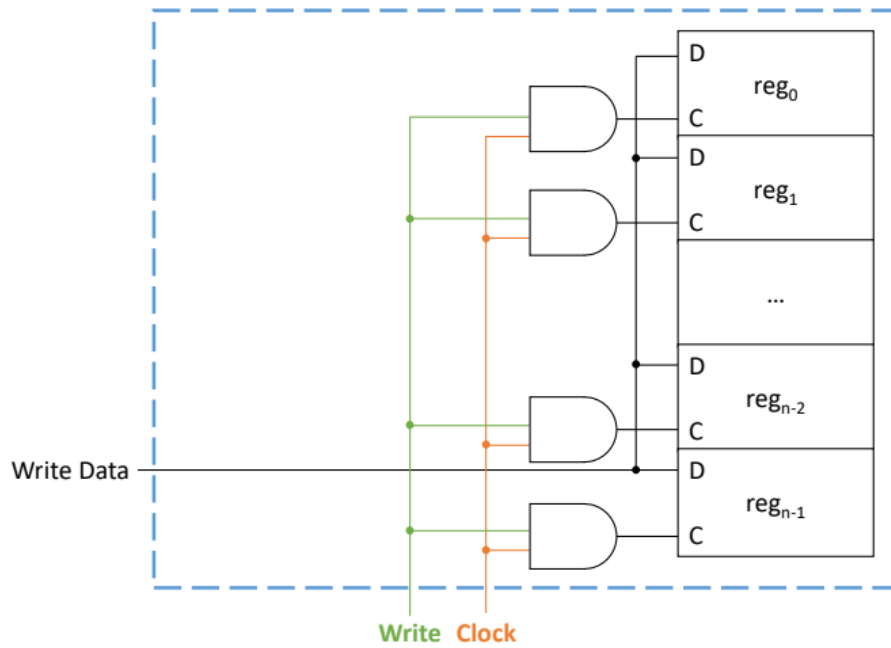
Writing Registers



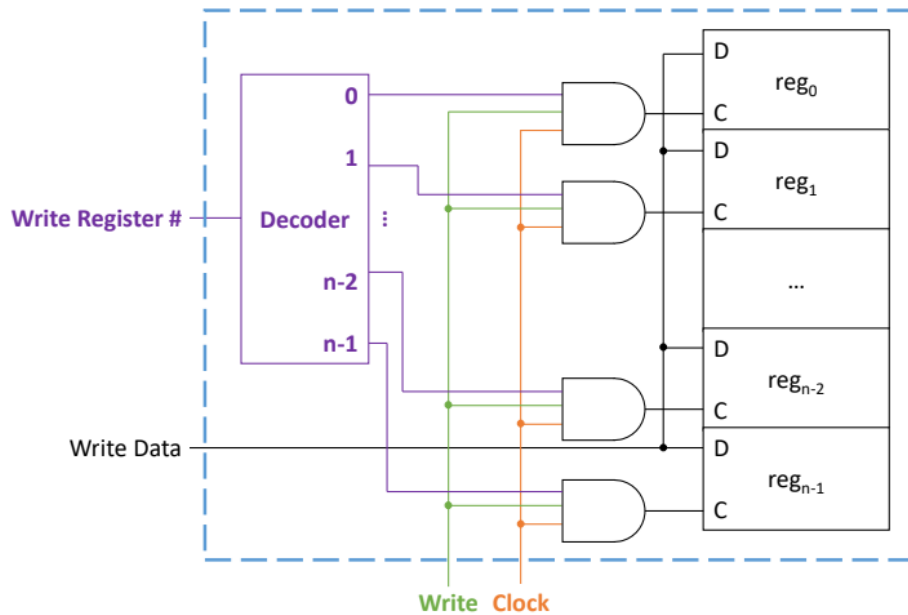
Writing Registers



Writing Registers



Writing Registers



7. Control Signals

Signal	Function
RegWrite	Enables writing data into a register.
ReadRegister1, ReadRegister2	Selects registers to read.
WriteRegister	Selects register to write into.
WriteData	Data to be written.

8. Example: 32-Register File with 5-bit Addressing

- Register file with 32 registers → Needs 5-bit address.
- Supports:
 - ReadRegister1 = 5 bits
 - ReadRegister2 = 5 bits
 - WriteRegister = 5 bits
 - WriteData = 32 bits
- Internally uses two 5-to-32 decoders and three 32-to-1 multiplexers.

9. Register Timing

- All operations are synchronized to a common **clock signal**.
- Data is latched into registers on rising/falling clock edges.
- Propagation delay must be considered during design.

10. Summary

Key Points

- Registers are the fastest memory units in CPU.
- Constructed using flip-flops and controlled using decoders and MUX.
- Read/write operations are controlled via specific signals.
- Register files allow simultaneous reading from multiple registers.